Smart Home Surveillance Controller System Using Verilog on Nexys A7 FPGA Board

Niveditha Binu
Department of Computer Science and Engineering
Indian Institute of Information Technology, Vadodara
Gandhinagar, India
202451179@iiitvadodara.ac.in

Priyanshu Virani
Department of Computer Science and Engineering
Indian Institute of Information Technology, Vadodara
Gandhinagar, India
202451169@iiitvadodara.ac.in

***Abstract*—The paper explains the design of a Smart Home Surveillance Controller System in Verilog HDL on the Nexys A7 FPGA development board. The system describes a simple home security system by keeping track of movement and the door with switches. The states of the system are controlled by FSM, such as Idle, Motion Detected, Door Alert, and Alarm. Depending on the input, the controller activates responses such as turning on camera indicators, blinking alarm LEDs or triggering an alarm. LEDs display the current status of the system and alerts. A 7-segment display provides real-time feedback of the current state of the system. The design was verified through behavioral simulation with all test cases passing, followed by hardware implementation on the Nexys A7 FPGA board.**

***Index Terms*—FPGA, Verilog HDL, Finite State Machine, Nexys A7, Home Security, Sequential Logic, Digital Design**

I. Introduction

The full form of FPGAs is Field Programmable Gate Arrays. They're integrated circuits and are programmed to perform specific hardware functions after manufacturing. Unlike traditional processors that execute software instructions sequentially, FPGAs allow designers to apply custom digital circuits that operate in parallel. This results in high speed as well as flexibility. FPGAs are mostly used in embedded systems, signal processing and control applications.

Home security systems are considered essential several homes, requiring a reliable and real-time monitoring and response techniques. From this project, we try to gain hands-on experience with Verilog HDL and hardware implementation by building a simplified security system using basic hardware components. Using an FPGA, the design can monitor several sensors simultaneously and produce outputs based on the current state of the system, a task ideally suited for hardware-level execution.

Our system tracks movement and opening of the door with input switches and responds with the help of alarms, a buzzer, and LED displays. All this is controlled by FSM that makes sure the controller works well in different states, given the different number of inputs. The design features motion, door, alarm, LED buttons, and a 7-segment display to provide instant feedback about the state. The entire implementation is in Verilog, following the simulation and hardware-testing.

Our paper is divided into seven sections: Section I introduces the FPGA. Section II talks about the Hardware Specifications and Interface. Section III describes the system architecture and design methodology. Section IV explains the FSM design and state transition logic. Section V covers all the module implementations performed in Verilog. Section VI presents the results performed on simulation and hardware, and Section VII concludes the paper.

II. Hardware Specifications and Interface

The Xilinx Artix-7 100T FPGA (XC7A100T-1CSG324C) with Nexys A7 board is used to run the project. This platform offers the high-performance logic needed and the support needed in real-time security applications.

A. Xilinx Artix-7 100T FPGA Architecture

The Artix-7 100T is optimised to use low power overhead. This architecture enables the Smart Home Surveillance Controller to perform in a highly parallel way.

B. Nexys A7 Peripheral Utilization

The system is based on the following components to simulate a comprehensive security environment:

- **Slide Switches**: Switches SW0 to SW2 are mapped to door monitoring inputs, motion detection, and arming.
- **Push Buttons**: The central and left buttons are used for reset and disarm overrides.
- **LED Status Indicators**: LEDs LD0-LD4 display a visual feedback for each state.
- **7-Segment Display**: A four-digit multiplexed display has textual status updates like ”IdLE” and ”ALrH”.
- **Pmod Header**: Pin JA1 is set to be the output of the system's audible buzzer.

C. I/O Standards and Signal Integrity

The constraints of *nexys_a7.xdc* ensure that the Nexys is reliable in its operation. The electrical properties of each port are clearly defined and indicated. All signals are configured to the LVCMOS33 standard, which further concludes that the logic levels used by 3.3V devices are the same throughout the hardware, avoiding communication problems or hardware destruction.

III. System Architecture and Design

The core of the Smart Home Surveillance Controller consists of a flexible architecture composed of five Verilog modules: *top_level, fsm, clk_divider, led_controller*, and *seg7_controller*. This structure simplifies the whole system, making the job of debugging and maintaining the code a lot easier. The design was implemented using Vivado 2025.1 and programmed onto the Nexys A7 FPGA board.

A. Interconnection of the Block Diagram and Module

The *top_level* module is the heart of this entire architecture, instantiating all the sub-modules and controlling the signal trace via internal wires. It then connects the physical input sensors and user controls to the central FSM logic and output. The architecture follows a systematic approach where the FSM regulates the behaviour of the system by switching between operational states depending on the sensor inputs.

To control the 100 MHz onboard clock that operates at a very high speed, the clock divider is divided into two signals: a 1 Hz clock for blinking LED warnings and a 1 kHz clock for seven-segment display multiplexing. The feedback is processed by highly-trained controllers. The LED controller controls the indicators and the buzzer, and the seven-segment controller shows textual information to the user.

B. Input and Output Mapping

The system uses three switches (SW0–SW2) that showcase control of the arming feature and sensor signals, and two push-button switches (BTNC and BTNL) for reset and disarm functions. The output consists of five LEDs (LD0–LD4), a buzzer connected to the Pmod JA header, and a 4-digit seven-segment display. All signals are assigned to designated physical pins of the Artix-7 FPGA which has been clearly explained in the table.

TABLE I
System Input/Output Mapping

Signal	Hardware	Pin	Function
clk	Oscillator	E3	100 MHz system clock
rst	BTNC	N17	Asynchronous reset to IDLE
disarm_btn	BTNL	P17	Disarm — force return to IDLE
arm	SW0	J15	Arms the surveillance system
motion	SW1	L16	Simulates PIR motion sensor
door	SW2	M13	Simulates door/window sensor
led_idle	LD0	H17	IDLE state indicator
led_motion	LD1	K15	Motion detected indicator
led_door	LD2	J13	Door alert indicator
led_alarm	LD3	N14	Alarm — blinks at 1 Hz
led_camera	LD4	R18	Camera active indicator
buzzer	Pmod JA1	C17	Continuous alarm buzzer

C. Theory of Operation and Data Flow

The Smart Home Surveillance Controller works through a sequential data process where sampling, processing, and displaying happen in real time. This kind of system is built in a way that allows handling many input data but keeps stable outputs.

- **Signal Acquisition**: The *top_level* module continuously monitors the physical state of switches SW0, SW1, and SW2, which simulate the arming control and environmental sensors.
- **Clock Synchronization**: The 100 MHz signal is fed into the *clk_divider*, which produces two lower-frequency pulses. The 1 kHz pulse ensures that the *seg7_controller* updates the display digits at a much faster rate, while the 1 Hz pulse provides the timing for the *led_controller* to pulse the alarm LED.
- **State Processing**: The *fsm* module acts as the heart of the entire system, evaluating the current state and sensor inputs on every rising edge of the system clock. Transitions are determined and a 2-bit state vector is given as output.
- **Output Generation**: The 2-bit state vector is transmitted to the *led_controller* and *seg7_controller*. These modules decode the vector into specific hardware responses: the *led_controller* activates the corresponding LEDs and buzzer, while the *seg7_controller* maps the state to character bit-patterns.
- **Safety Overrides**: The *disarm_btn* (BTNL) is routed directly into the FSM combinational logic. This provides a switch that bypasses standard transition rules to put the system back into the safe state.

In this Smart Home Surveillance Controller System, the overall architecture is modular in nature and consists of 5 main components: top-level module, FSM controller, clock divider, LED controller, and seven-segment display controller. All of them are interconnected. This is to get the expected functionality of monitoring inputs and generate appropriate outputs.

The top-level module acts as the integration unit. It connects all submodules and manages the input and output signals. The FSM module forms the core of the system. FSM modules handle state transitions based on input such as motion and door signals. To generate a slower clock signal from the main FPGA, the clock divider module is used. It is used to ensure stable operation and visible output transitions. For displaying system status, an LED controller is used. The seven segment controller is used to display state information if required.

Firstly, the system takes input from switches which simulate sensors. These inputs are processed by the FSM. FSM determines the current state of the system and based on the state, appropriate outputs are generated and displayed through LEDs.

The main functionality of the Smart Home Surveillance Controller System is governed by a FSM (finite state machine). The responsibility of controlling the behaviour of the system by transitioning between different operational states based on input conditions is done by FSM. A synchronous design approach is adopted, where all state transitions occur on the rising edge of the clock signal, ensuring stable and predictable operation.

The system is made using a four state FSM. These include: Idle, Motion Detection, Intrusion, and Alarm. While the system is in idle and doesn't take any action, it continues to monitor any input signals continuously. The system will move to the Motion Detection state whenever an input switch detects motion. At that point, motion has been detected, but a threat has not yet been validated.

But if the door sensor is activated, the system transitions to the Intrusion state. This represents a higher level of alert, where unauthorized access is detected. Upon confirming intrusion conditions, the FSM moves to the Alarm state, where the system displays visual alerts such as LEDs to indicate a security detection. The Alarm state will continue to trigger until a reset signal is applied or the system is manually returned to the Idle state.

State machine is formed by merging a few inputs to cause state transitions using conditional expressions within the "always" block written in Verilog code. State variables are encoded in binary form to minimise the amount of resources and provide a more straightforward implementation for the FPGA synthesis process.

The FSM operates as a Moore machine, where the outputs depend primarily on the current state without directly depending on the inputs. In other words, behavior of the output becomes predictable since the FSM will generate the appropriate signal according to the particular state. For example, the system will sound the alarm only when being at the Alarm state, irrespective of minor variations in inputs.

In addition, FSM provides a practical solution for implementing the logic of a surveillance system. It ensures proper responses to various situations and state transitions, making it an excellent choice for the hardware implementation.

IV. Finite State Machine Design

FSM is implemented using a standard two-always-block Verilog style. This further separates sequential logic from combinational logic to ensure the proper working of the system.

A. Logic Structure

Two functional blocks are used to describe the structure of FSM:

- **Sequential Block**: This block operates at a clock frequency of 100 MHz, and is responsible for updating the state register. It includes an asynchronous reset that forces the system to the IDLE state upon assertion.
- **Combinational Block**: This block showcases the next-state logic based on current state and environmental inputs. The *disarm_btn* signal is given the highest priority and is evaluated before the case statement to confirm the possibility of an override.

B. State Encoding and Output Mapping

The system defines four operational states encoded as 2-bit values to optimize utilization of hardware resources on the Artix-7 FPGA.

TABLE II
FSM State Descriptions and Outputs

State	Encoding	Condition to Enter	LEDs	7-Seg
IDLE	2'b00	Reset or no sensors	LD0	IdLE
MOTION_DET	2'b01	arm=1, motion=1	LD1, LD4	Motn
DOOR_ALERT	2'b10	arm=1, door=1	LD2	door
ALARM	2'b11	Sensor escalation	LD3, LD4	ALrH

C. Transition Rules and Design Decisions

State escalation paths are designed to cover all threat scenarios:

- **Direct Escalation**: There's a path from IDLE to ALARM if both sensors trigger simultaneously while the system is armed.
- **Sequential Escalation**: The system moves from MOTION_DET to ALARM if a door opens during active motion, or from DOOR_ALERT to ALARM if motion is detected while a door is open.
- **Sticky Alarm**: The ALARM state is designed to transition back to itself, also called as a "sticky" state. This ensures that the alarm does not self-clear; only user intervention via the disarm button or reset can return the system to IDLE.

V. Verilog Module Implementation

The implementation of the Smart Home Surveillance Controller is designed using a hardware approach, giving each part of the system to perform a specific function, thereby ensuring scalability and making the design easier to understand, test, and debug.

The architecture is revolved around a top-level module that connects all the sub-modules together.

We first have a clock divider that's used to generate slower clock signals from the main 100 MHz clock, which help in producing outputs like blinking LEDs and stable display on the 7-segment.

FSM controls how the system behaves based on intrusion, motion detection, arm, disarm and reset.

The output of the FSM is connected to LED controller, which shows the status, the buzzer, which indicates the alarm, and the 7-segment display controller, which shows the current state.

To conclude, each and every signal is mapped to the physical FPGA pins using the constraints (.xdc) file, ensuring a stable connection between the design and the hardware board.

This modular design helps organize the system properly, making it easy to manage and extend in the future.

A. top_level.v

The top module connects all the sub modules together to generate a proper working system. It links the physical FPGA pins to all these sub modules. It takes inputs from the FSM and switches, passes them to the FSM, and connects the outputs to LEDs, buzzer, and display. Internal wires such as *clk_1hz*, *clk_1khz* and a 2-bit state signal to pass the data between the sub-modules. This ensures that the overall design is organized.

B. *clk_divider.v*

This sub module was created, considering the fact that the Nexys A7 runs at a very fast 100 MHz. The clock divider slows this down. For 1hz signal, a 26-bit counter was used to toggle a signal every 50,000,000 cycles, thus providing the blinking sensation needed for the alarm. For the 1khz signal, we used a 17-bit counter. This is used to refresh the 7-segment display digits at a rapid rate.

C. *fsm.v*

This is the heart of our entire design. Here, we've chosen two blocks, one for sequential and another for combinational, so that the synthesis can be more efficient. The sequential block updates the current state on every clock pulse, whereas the combinational block decides the next state based on inputs such as motion, door and arm. This has been specifically designed so that disarm_btn forces the system to return to IDLE.

D. *led_controller.v*

This module displays the system output, such as lights and sound. *case* statement is used to define what's meant to happen at each state. The always @(*) block maps each FSM state to its corresponding output signals. During the ALARM state, we've assigned *led_alarm* = *clk_1hz*. This makes the red LED blink automatically at a steady 1 Hz frequency. We also made sure that the buzzer and led_camera stay active during the alarm to provide a full alert sensation.

E. *seg7_controller.v*

This module has been created to show status words. This has been used to manage digits on the board. We defined custom bit patterns for characters like "I", "d", "L", and "E", but since they share the same segment lines, we've used **time-division multiplexing**.

F. *nexys_a7.xdc*

The constraint file is the connection between the software and hardware. This file guarantees that toggling SW0 will send the arm command and when the alarm is triggered, LD3 will light up.

VI. Results

The Smart Home Surveillance System was tested using both simulation and hardware implementation. The simulation in Vivado validated the FSM logic's correctness. The Nexys A7 FPGA board performed hardware testing. The results confirm that the system behaves as expected under different input conditions.

A. *Simulation Results*

The functionality of the system was verified by running the behavioral simulation in Vivado. It depicts correct state transitions of the FSM based on several input conditions such as intrusion, motion detection, door alert, reset signals etc.

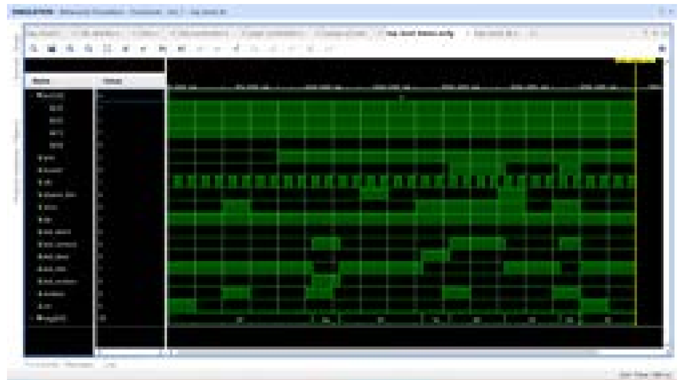

Fig. 1. Simulation Waveform showing FSM Transitions under several input conditions

The waveform represents multiple test scenarios that are executed in a sequential manner. The clock signal is continuously running. But several inputs such as disarm_btn, door and motion are varied with respect to time.

From the given waveform, we can clearly conclude using the following observations.

1) The system only begins in the IDLE state when all the inputs are inactive.
2) On activation of the motion signal the FSM switches to the MOTION_DET state and the appropriate LED outputs are altered.
3) Upon triggering the door signal, the system enters the DOOR_ALERT state.
4) The system goes to the ALARM state when both motion and door signals are running simultaneously.
5) The reset system brings the system back to the IDLE state immediately.

The 7-segment signal validates the output based on the current state, thereby ensuring correct display behavior.

B. *Hardware Results*

The hardware logic was tested on the Nexys A7 FPGA board to validate real-time performance. Based on the results, we conclude that the system behaves as expected under several input conditions using switches and push buttons. Each test case was verified by observing LED outputs and 7-segment display responses.

1) Test Case 1: IDLE State: In this test case, all the inputs: arm, motion and door are set to 0. The system was reset, and this shows that the FSM remains in the IDLE state.

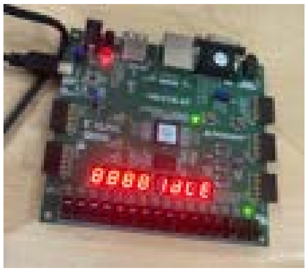

Fig. 2. Hardware output showing IDLE state

LED LD0 remained ON, indicating that the system is in the IDLE state, and the 7-segment display showed ”IdLE”.

2) Test Case 2: Armed State (No Sensor Trigger): In this test case, the system was armed by setting the arm switch (SW0) to HIGH while keeping all sensor inputs inactive.

Fig. 3. Hardware output showing Armed state with no sensor activity

Even after arming, since no intrusion was detected, the system remained in the IDLE state. LD0 remained ON and the 7-segment display continued to show ”IdLE”. This confirms that the system correctly waits for sensor input before transitioning states.

3) Test Case 3: Motion Detection: With the system armed (SW0 HIGH), the motion input (SW1) was activated.

Fig. 4. Hardware output showing Motion Detection state

The FSM transitioned to the MOTION_DET state. LD1 was activated to indicate motion detection, and LD4 was turned ON to represent camera activation. The 7-segment display showed ”Motn”. Once the motion input was removed, the system automatically returned to its IDLE state.

4) Test Case 4: Door Alert: With the system armed, the door input (SW2) was triggered while motion remained inactive.

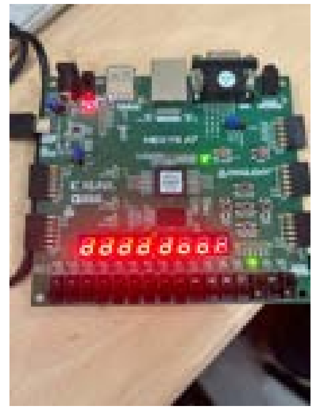

Fig. 5. Hardware output showing Door Alert state

The FSM transitioned to the DOOR_ALERT state. LD2 got turned ON indicating door intrusion, and the 7-segment display showed ”door”. The system returned to IDLE once the door input was cleared.

5) Test Case 5: Alarm State: The ALARM state was verified under multiple conditions to ensure correctness of the FSM design.

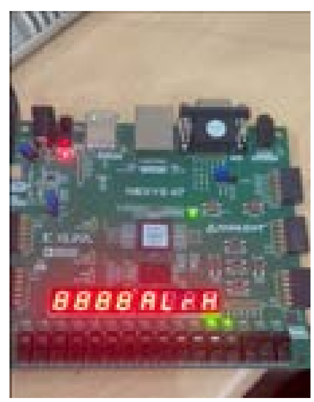

Fig. 6. Hardware output showing Alarm state

The system entered the ALARM state under the following conditions:

- Motion followed by door activation
- Door followed by motion activation
- Simultaneous activation of both motion and door inputs

The alarm state was signaled by blinking of LD3 at a rate of 1 Hz. LD4, which is the camera indicator, lit up while the 7-segment display showed “ALrH.”

The system could only return to the IDLE state using the disarm button (BTNL) or reset button (BTNC), confirming correct priority handling in the FSM.

6) Test Case 6: Door followed by Motion: The system was first armed (SW0 HIGH) after which the door switch (SW2) was activated bringing the system to the DOOR ALERT state. The motion switch (SW1) was also activated.

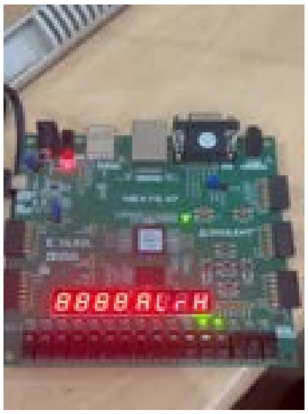

Fig. 7. Hardware output showing Door followed by Motion

From both sources, it's evident that the FSM changed its state to ALARM. The LD3 began to blink, indicating an alarm. It means that the system is able to manage the alarm in case of any input trigger, regardless of whether the motion sensor or door sensor was the trigger. The reset signal using BTNC returned the system to its Idle state.

7) Test Case 7: Direct Alarm Trigger: In this test case, the system was armed and both motion (SW1) and door (SW2) inputs were activated simultaneously.

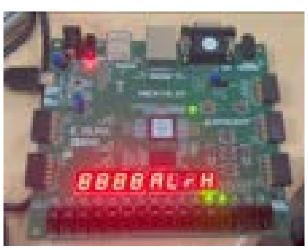

Fig. 8. Hardware output showing Direct Alarm Trigger

The FSM directly transitioned from the IDLE state to the ALARM state, bypassing MOTION_DET and DOOR_ALERT. LD3 started blinking immediately, and the 7-segment display showed "ALrH".

This confirms that the system prioritizes simultaneous intrusion detection and responds instantly to high-risk conditions. The system was returned to IDLE using BTNL.

VII. Conclusion

This project has clearly demonstrated the simulation and hardware logic of a Smart Home Surveillance Controller on the Nexys A7 FPGA using Verilog HDL. The system successfully displayed four-state Finite State Machine (FSM) operation with state transitions, LED and buzzer output responses, and 7-segment display feedback.

The simulation logic was correctly tested by showing a correct waveform behaviour and how state transitions happen under various input conditions. And the hardware logic displayed a real-time display using switches, LEDs, and 7-segment display.

Proper functionality was shown on several test cases, from IdLe operation to motion detection, door alerts, alarm escalation, and direct alarm triggering. The modular Verilog architecture showed clear separation of logic and clear resource utilization.

This project effectively demonstrates key digital design concepts such as FSM design using the two-always-block methodology, clock division for timing control, combinational output logic, and 7-segment display multiplexing.

In a nutshell, we can conclude that the implementation displays the effectiveness of FSM-based design in real-world systems and provides a strong foundation for future enhancements such as integration with real sensors, real-time hardware monitoring etc.

Acknowledgment

We want to extend our deepest gratitude to our professor Dr. Sanjeev Yadav, and our teaching assistant, Jitu Sir, for their valuable guidance and support throughout the making of this project. We would also like to acknowledge the department of electrical engineering at IIIT Vadodara, who helped us by providing all the necessary resources and devices such as Nexys A7 FPGA Board. Lastly, we would like to thank all those people who had a direct or indirect contribution towards completing this project.

References

[1] Digilent Inc., "Nexys A7 Reference Manual," 2023. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[2] Xilinx Inc., "Vivado Design Suite User Guide: Synthesis," UG901, 2023.

[3] M. Morris Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Pearson, 2012.

[4] J. Bhasker, A Verilog HDL Primer, 3rd ed. Star Galaxy Publishing, 2005.

[5] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.